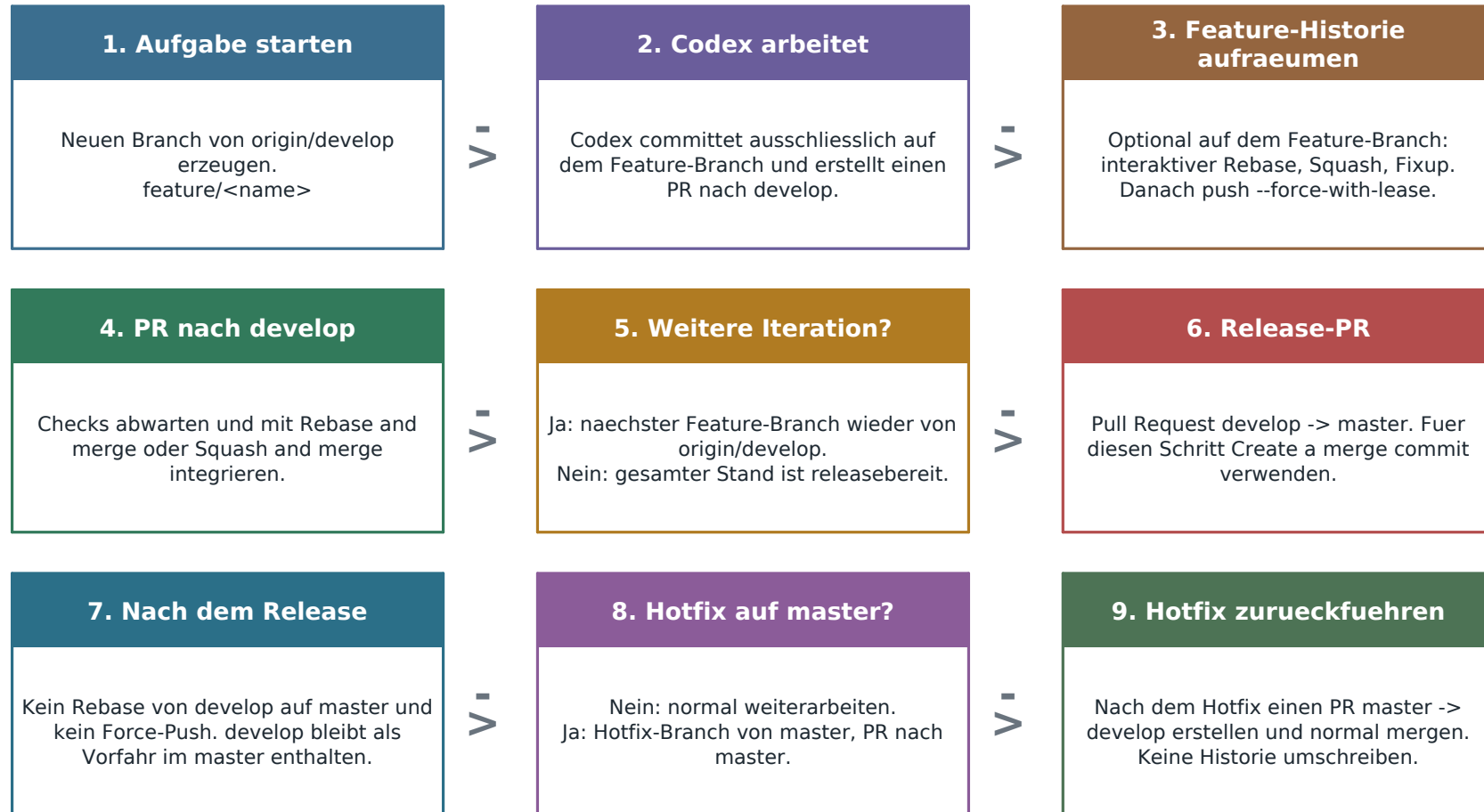


Empfohlener Git-Workflow mit geschuetzten Branches

Ziel: Codex arbeitet ueber Feature-Banches und Pull Requests auf develop. Fertige Entwicklungsstaende gelangen per Pull Request nach master. Die geschuetzten Branches werden niemals lokal umgeschrieben.



Wichtigste Regel: Rebase, Squash und Force-Push sind nur auf kurzlebigen, ungeschuetzten Feature-Banches erlaubt. master und develop werden nur durch Pull Requests vorwaerts bewegt.

Entscheidungsbaum

Frage	Ja	Nein
Beginnt eine neue Codex-Aufgabe?	Feature-Branch von origin/develop anlegen und PR nach develop.	Naechste Frage.
Muss die Commit-Historie bereinigt werden?	Nur den Feature-Branch interaktiv rebasen/squashen; danach git push --force-with-lease.	PR unveraendert weiterfuehren.
Ist nur ein einzelnes Teilfeature fertig?	PR nach develop mergen; danach neuer Feature-Branch von aktuellem develop.	Naechste Frage.
Ist der gesamte Stand releasebereit?	PR develop -> master; Merge-Commit verwenden, damit develop Vorfahr von master bleibt.	Weiter auf develop iterieren.
Gab es einen Hotfix direkt fuer master?	Nach dem Master-PR einen Rueckfuehrungs-PR master -> develop erstellen.	develop muss nicht kuenstlich auf master rebased werden.
Will Git fuer develop einen Force-Push?	Abbrechen. Das ist ein Zeichen, dass gerade die Historie des geschuetzten Branches umgeschrieben wird.	Normaler Push beziehungsweise PR ist korrekt.

Warum beim Release ein Merge-Commit?

Wenn GitHub den PR develop -> master per Rebase and merge integriert, erzeugt GitHub neue Commit-IDs auf master. Die urspruenglichen Commits auf develop sind dann keine Vorfahren dieser neuen Commits. Ein anschliessendes Rebase von develop auf master schreibt develop um und verlangt einen Force-Push. Ein normaler Merge-Commit bewahrt dagegen die Abstammung: develop bleibt in master enthalten.

Alternative, noch einfacher: Falls du die dauerhafte Zwischenstufe develop nicht wirklich brauchst, koennen Codex-Feature-Branche direkt PRs nach master stellen. Dann entfaellt das gesamte Synchronisationsproblem. Fuer dein beschriebenes Modell ist der oben gezeigte Merge-Commit beim Release jedoch die kleinste Aenderung.

Konkrete Befehle und GitHub-Einstellungen

Neue Aufgabe beginnen

```
git fetch origin
git switch develop
git pull --ff-only origin develop
git switch -c feature/meine-aufgabe
```

Feature-Branch aufräumen

```
git fetch origin
git rebase -i origin/develop
git push --force-with-lease origin feature/meine-aufgabe
```

Branch	Schutz	Merge-Methode
master	PR und Statuschecks erforderlich; Force-Push und Loeschen blockiert.	Release develop -> master: Create a merge commit.
develop	PR und Statuschecks erforderlich; Force-Push und Loeschen blockiert.	Feature-PRs: Rebase and merge oder Squash and merge.
feature/*	Nicht schuetzen; normaler Push durch Codex und dich.	Rebase/Squash erlaubt; danach --force-with-lease.

Stoppsignal: Sobald Git fuer master oder develop einen Force-Push verlangt, wird gerade die Historie eines geschuetzten Branches umgeschrieben. Vorgang abbrechen. Hotfixes werden per PR master -> develop zurueckgefuehrt.